

```

<script lang="ts">
  import { invoke } from "@tauri-apps/api/core";
  import { open } from "@tauri-apps/plugin-dialog";
  import { listen } from "@tauri-apps/api/event";
  import { onMount, tick } from "svelte";

  // --- PERSISTENCE & STATE ---
  let baseUrl = $state(localStorage.getItem("raa_base_url") || "");
  let modelName = $state(localStorage.getItem("raa_model_name") || "");
  let isConfigured = $derived(baseUrl.trim().length > 0 && modelName.trim().length > 0);
  let isDev = $state(false);

  $effect(() => {
    localStorage.setItem("raa_base_url", baseUrl);
    localStorage.setItem("raa_model_name", modelName);
  });

  let activeTab = $state("welcome");
  let isProcessing = $state(false);
  let currentReport = $state({ verdict: "", reasoning: "", target_name: "", is_error: false });
  let certMsg = $state("");
  let commandInput = $state("");
  let ledgerContent = $state("");
  let integrityReport = $state<any>(null);

  // --- MONITOR STATE ---
  let allowedExts = $state([".md", ".js", ".yaml", ".zip", ".env", ".txt"]);
  let activeFiles = $state<string[]>([]);
  let skippedFiles = $state<string[]>([]);
  let activeScrollContainer: HTMLElement | undefined = $state();
  let skippedScrollContainer: HTMLElement | undefined = $state();

  async function scrollToBottom(el: HTMLElement | undefined) {
    if (el) { await tick(); el.scrollTop = el.scrollHeight; }
  }

  onMount(() => {
    isDev = window.location.hostname === 'localhost';
    let unlisten: any;
    async function setupListener() {
      unlisten = await listen("scan-event", (event: any) => {
        const { path, status } = event.payload;
        if (status === "Active") {
          activeFiles = [...activeFiles, path];
          scrollToBottom(activeScrollContainer);
        } else {
          skippedFiles = [...skippedFiles, path];
          scrollToBottom(skippedScrollContainer);
        }
      });
    }
    setupListener();
    return () => { if (unlisten) unlisten(); };
  });

  // --- UI PARSING LOGIC ---
  function resetResults() {
    currentReport = { verdict: "", reasoning: "", target_name: "", is_error: false };
    certMsg = ""; activeFiles = []; skippedFiles = [];
  }

```

```

}

function getReportSegments(text: string) {
    if (!text) return [];
    let segments = text.split(/(?:\n|^)(?=\d+\.\.|\*\*Item\Item\s\d+:|---\n)/g)
        .filter(s => s.trim().length > 5);

    if (segments.length === 0 && text.trim().length > 0) {
        segments = [text];
    }
    console.log("Segments from reasoning:", segments);
    return segments;
}

function highlightSegment(text: string) {
    if (!text) return "";
    return text
        .replace(/([\w\-\_]+\.(?:md|rs|ts|js|py|yaml|zip|toml|json|txt|sh))/gi, '
<span class="path-text">$1</span>')
        .replace(/(SAFE|CERTIFIED|CLEAN|ALL SAFE)/g, '<span class="text-success">$
1</span>')
        .replace(/(VIOLATION|THREAT|DANGER|MALICIOUS|DETECTED)/g, '<span class="te
xt-danger">$1</span>');
}

// --- HANDLERS ---
async function handleAudit(e?: Event) {
    if (e) e.preventDefault();
    if (!commandInput.trim()) return;
    resetResults();
    isProcessing = true;
    try {
        // ADDED: <any> to tell TypeScript the incoming data is valid
        const report = await invoke<any>("audit_command", { commandStr: commandInp
ut, baseUrl, modelName });
        currentReport = report;
        await tick();
        console.log("Updated currentReport:", currentReport);
    }
    catch (err) {
        currentReport = { verdict: "Error", reasoning: String(err), target_name: "
Audit", is_error: true };
        await tick();
    }
    finally { isProcessing = false; }
}

async function handleBrowseFile() {
    resetResults();
    try {
        const selected = await open({ multiple: true });
        if (selected) {
            isProcessing = true;
            const paths = Array.isArray(selected) ? selected : [selected];
            currentReport = await invoke("scan_file_integrity", { filePaths: paths,
baseUrl, modelName });
        }
        catch (err) { currentReport = { verdict: "Error", reasoning: String(err),
target_name: "Files", is_error: true }; }
        finally { isProcessing = false; }
}

```

```

}

async function handleBrowseArchive() {
  resetResults();
  try {
    const selected = await open({ multiple: false, filters: [{ name: 'Archives', extensions: ['zip'] }] });
    if (selected && !Array.isArray(selected)) {
      isProcessing = true;
      currentReport = await invoke("scan_compressed_archive", { zipPath: selected, allowedExtensions: allowedExts, baseUrl, modelName });
    } catch (err) { currentReport = { verdict: "Error", reasoning: String(err), target_name: "Archive", is_error: true }; }
    finally { isProcessing = false; }
  }

  async function handleCertifyFolder() {
    resetResults();
    try {
      const selectedFolder = await open({ directory: true, multiple: false });
      if (selectedFolder) {
        isProcessing = true;
        certMsg = await invoke("generate_manifest", { folderPath: selectedFolder, allowedExtensions: allowedExts, baseUrl, modelName });
      } catch (err) { certMsg = String(err); }
      finally { isProcessing = false; }
    }

    async function loadLedger() { activeTab = 'ledger'; ledgerContent = await invoke("read_ledger"); }
    async function runIntegrityCheck() { activeTab = 'integrity'; integrityReport = await invoke("check_integrity"); }
  }
</script>

<div class="app-layout" class:scanning={isProcessing}>
  <header class="top-bar">
    <button type="button" class="logo-btn" onclick={() => activeTab = 'welcome'}>
  >

```